Tema:

Desenvolver uma aplicação com o objetivo de aplicar, na prática, os principais conceitos de Programação Orientada a Objetos (POO) por meio da implementação de uma API REST utilizando TypeScript.

A atividade visa consolidar conhecimentos fundamentais como abstração, encapsulamento, herança e polimorfismo, além do uso de interfaces, classes abstratas e generics.

Requisitos de Avaliação

- Valor: 0,25 pts. na média final.
- Forma de entrega: individual ou em dupla.
- Data de entrega: 23/03/2026 – os alunos ausentes podem entregar dias 24 ou 25/03/2026.

Descrição geral

O aluno deverá implementar uma API REST responsável por manipular três estruturas de dados lineares genéricas:

- Pilha (Stack)
- Fila (Queue)
- Lista (List)

A aplicação deverá manter os dados **em memória**, sem utilização de banco de dados.

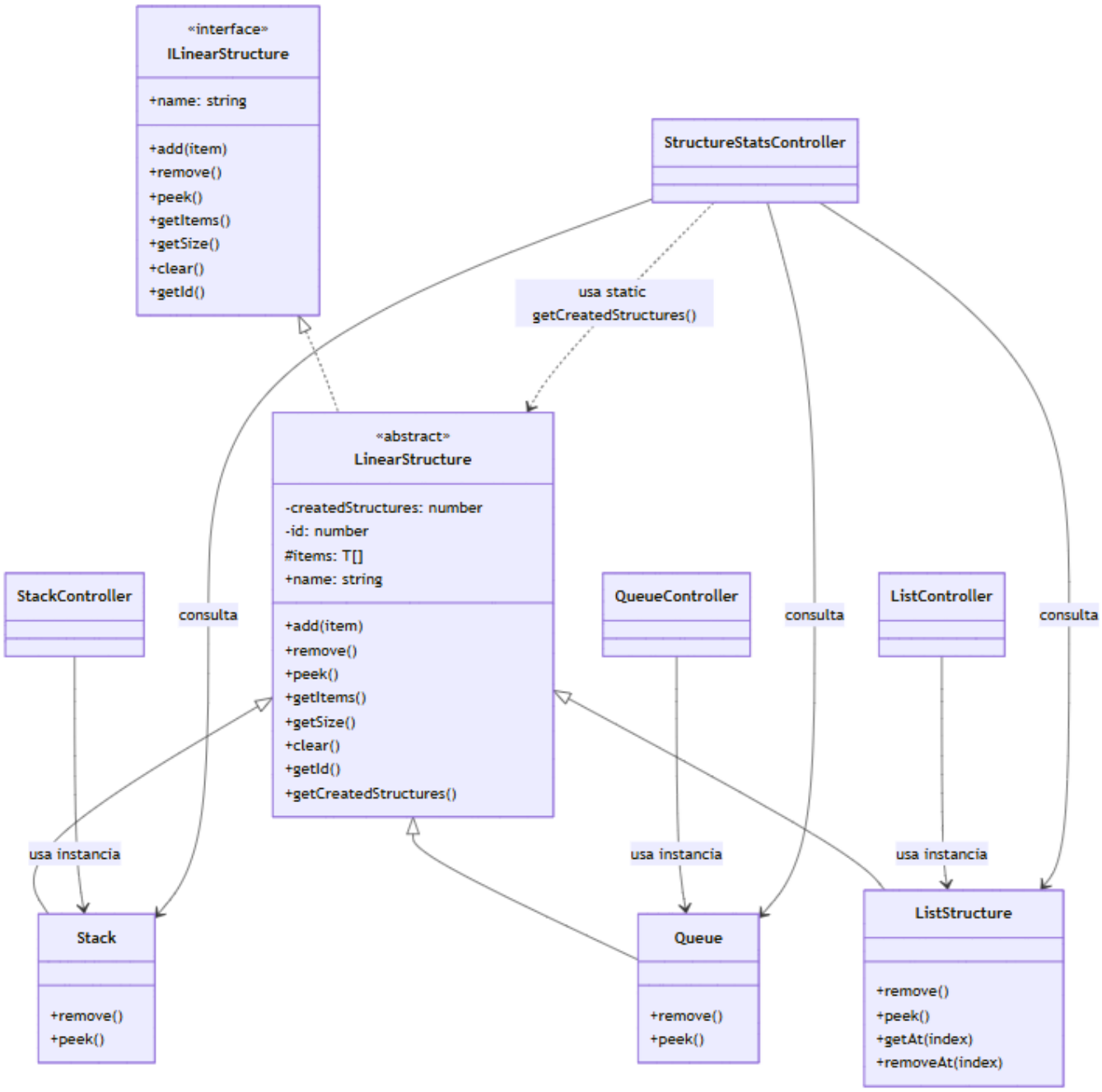
A API deverá disponibilizar endpoints para inserção, remoção, consulta e listagem dos elementos de cada estrutura.

Requisitos de implementação

Diagrama de classes

A aplicação deve ter as seguintes classes e interfaces.

Observação: T representa o tipo genérico armazenado na estrutura.



Conceitos de POO no projeto

- Classe: Stack, Queue e ListStructure representam implementações concretas de estruturas lineares.
- Interface: ILinearStructure define o contrato comum que todas as estruturas devem seguir.
- Classe abstrata: LinearStructure concentra comportamento compartilhado e impede instanciação direta.
- Herança: Stack, Queue e ListStructure estendem LinearStructure.
- Polimorfismo: as três estruturas compartilham a mesma interface pública (add, remove, peek, getItems, getSize, clear), mas cada uma implementa comportamentos próprios para remoção e consulta.
- Encapsulamento: o projeto utiliza private, protected e public para controlar acesso ao estado interno.
- static: LinearStructure.getCreatedStructures() contabiliza quantas estruturas foram instanciadas.

- Classe genérica: as estruturas usam <T> para armazenar qualquer tipo de dado.

Propriedades e métodos

ILinearStructure<T>

Propriedade:

- name: nome público da estrutura.

Métodos:

- add(item: T): adiciona um item na estrutura.
- remove(): remove um item segundo a regra da estrutura concreta.
- peek(): consulta o item que seria removido primeiro, sem removê-lo.
- getItems(): retorna uma cópia dos itens armazenados.
- getSize(): retorna a quantidade de itens.
- clear(): remove todos os itens.
- getId(): retorna o identificador da estrutura.

LinearStructure<T>

Propriedades:

- createdStructures: propriedade private static usada para contar quantas estruturas foram criadas.
- id: propriedade private readonly que guarda o identificador único da instância.
- items: propriedade protected que armazena internamente os elementos da estrutura.
- name: propriedade public readonly com o nome da estrutura.

Métodos:

- constructor(name: string): define o nome da estrutura, incrementa o contador estático e gera o id.
- add(item: T): adiciona um item ao array interno items.
- remove(): método abstrato; cada subclasse define como o item é removido.
- peek(): método abstrato; cada subclasse define qual item é consultado.
- getItems(): devolve uma cópia do array interno para evitar alteração externa direta.
- getSize(): informa o tamanho atual da estrutura.
- clear(): limpa todos os itens armazenados.
- getId(): retorna o identificador da instância.
- getCreatedStructures(): método static que retorna o total de estruturas criadas.

Stack<T>

Métodos:

- constructor(name = "Pilha"): cria uma pilha com nome padrão ou personalizado.

- remove(): remove e retorna o item do topo da pilha com pop().
- peek(): retorna o item do topo sem removê-lo.

Queue<T>

Métodos:

- constructor(name = "Fila"): cria uma fila com nome padrão ou personalizado.
- remove(): remove e retorna o primeiro item da fila com shift().
- peek(): retorna o primeiro item da fila sem removê-lo.

ListStructure<T>

Métodos:

- constructor(name = "Lista"): cria uma lista com nome padrão ou personalizado.
- remove(): remove e retorna o último item da lista com pop().
- peek(): retorna o último item da lista sem removê-lo.
- getAt(index: number): retorna o item do índice informado ou undefined se o índice for inválido.
- removeAt(index: number): remove e retorna o item do índice informado ou undefined se o índice for inválido.

Endpoints

Pilha

- POST /api/pilha adiciona um item ao topo da pilha
- DELETE /api/pilha remove o item do topo
- GET /api/pilha/topo consulta o item do topo
- GET /api/pilha lista os itens da estrutura
- DELETE /api/pilha/limpar remove todos os itens

Exemplos com curl:

```
curl -X POST http://localhost:3001/api/pilha \
  -H "Content-Type: application/json" \
  -d '{"item":"Livro"}'
curl -X GET http://localhost:3001/api/pilha
curl -X GET http://localhost:3001/api/pilha/topo
curl -X DELETE http://localhost:3001/api/pilha
curl -X DELETE http://localhost:3001/api/pilha/limpar
```

Fila

- POST /api/fila adiciona um item ao final da fila

- DELETE /api/fila remove o item da frente
- GET /api/fila/frente consulta o item da frente
- GET /api/fila lista os itens da estrutura
- DELETE /api/fila/limpar remove todos os itens

Exemplos com curl:

```
curl -X POST http://localhost:3001/api/fila \
  -H "Content-Type: application/json" \
  -d '{"item":"Cliente 1"}'
curl -X GET http://localhost:3001/api/fila
curl -X GET http://localhost:3001/api/fila/frente
curl -X DELETE http://localhost:3001/api/fila
curl -X DELETE http://localhost:3001/api/fila/limpar
```

Lista

- POST /api/lista adiciona um item ao final da lista
- DELETE /api/lista remove o último item
- GET /api/lista/ultimo consulta o último item
- GET /api/lista lista todos os itens
- GET /api/lista/:index consulta um item pelo índice
- DELETE /api/lista/:index remove um item pelo índice
- DELETE /api/lista/limpar remove todos os itens

Exemplos com curl:

```
curl -X POST http://localhost:3001/api/lista \
  -H "Content-Type: application/json" \
  -d '{"item":"Tarefa A"}'
curl -X GET http://localhost:3001/api/lista
curl -X GET http://localhost:3001/api/lista/ultimo
curl -X GET http://localhost:3001/api/lista/0
curl -X DELETE http://localhost:3001/api/lista
curl -X DELETE http://localhost:3001/api/lista/0
curl -X DELETE http://localhost:3001/api/lista/limpar
```

Estatísticas

- GET /api/estatisticas retorna a quantidade total de estruturas criadas e o estado das instâncias em uso

Exemplo com curl:

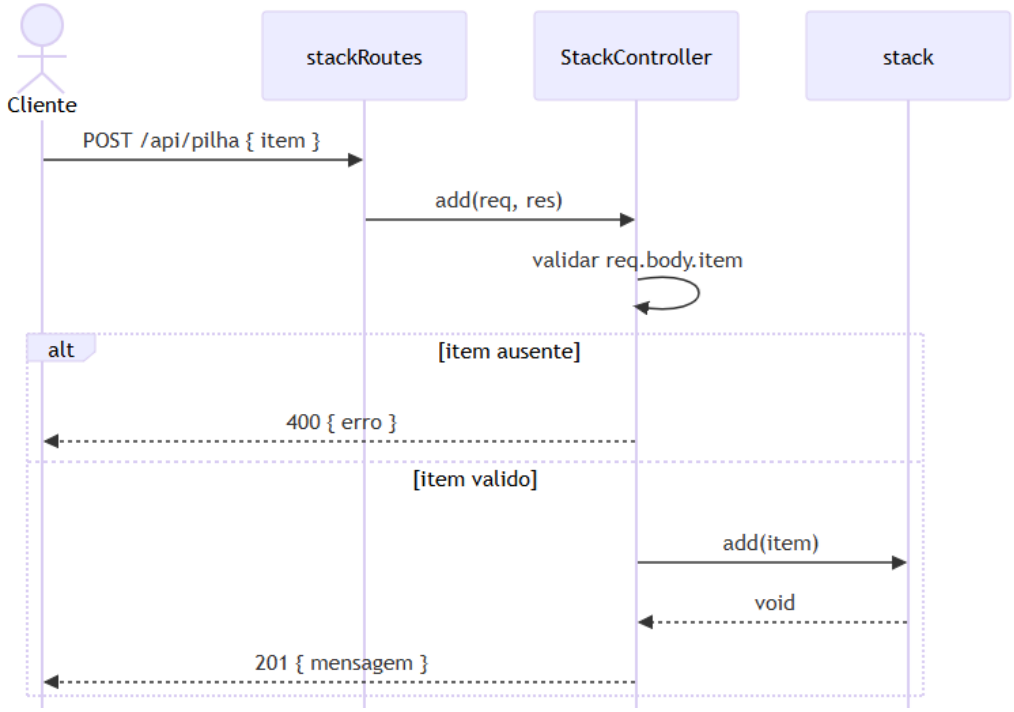
```
curl -X GET http://localhost:3001/api/estatisticas
```

Organização do projeto

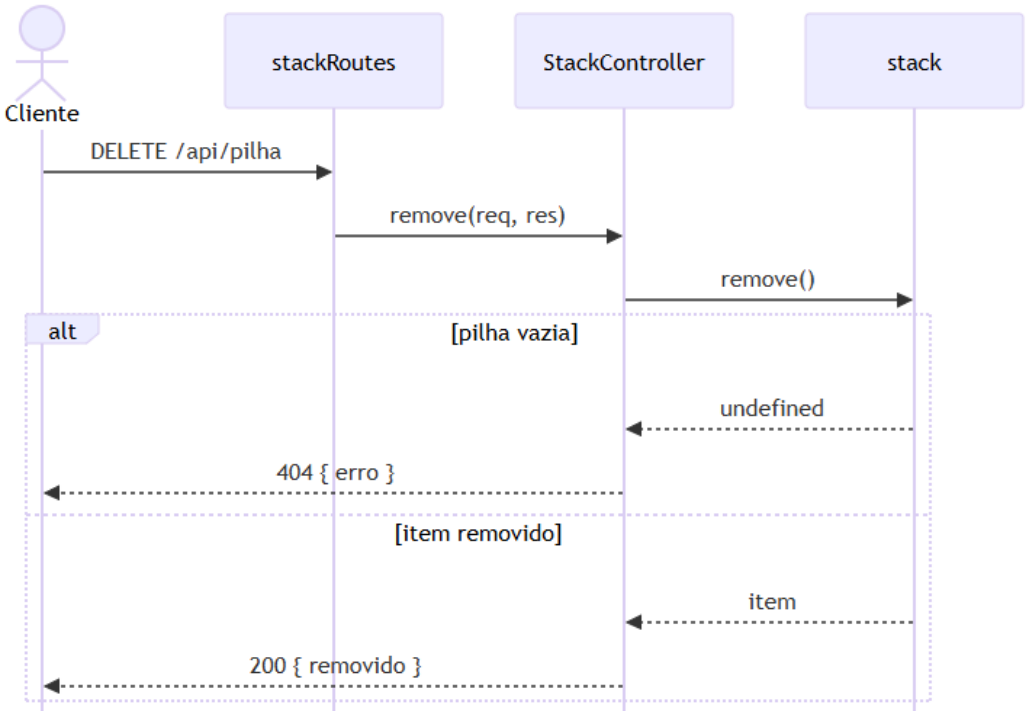
- src/models: classes base e estruturas de dados (LinearStructure, Stack, Queue, ListStructure)
- src/models/interfaces: contrato principal das estruturas (ILinearStructure)
- src/models/instances.ts: instâncias compartilhadas de pilha, fila e lista
- src/controllers: camada responsável por tratar requisições e respostas HTTP
- src/routes: definição das rotas da API
- src/server.ts: inicialização do servidor Express e carregamento das variáveis de ambiente

Diagramas de sequência por rota

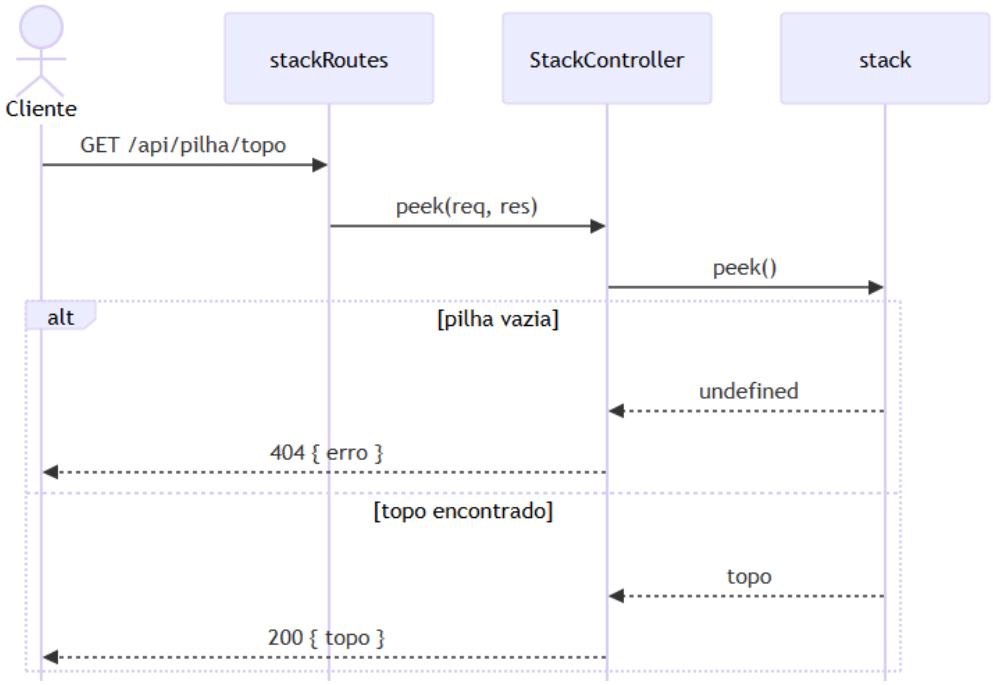
POST /api/pilha



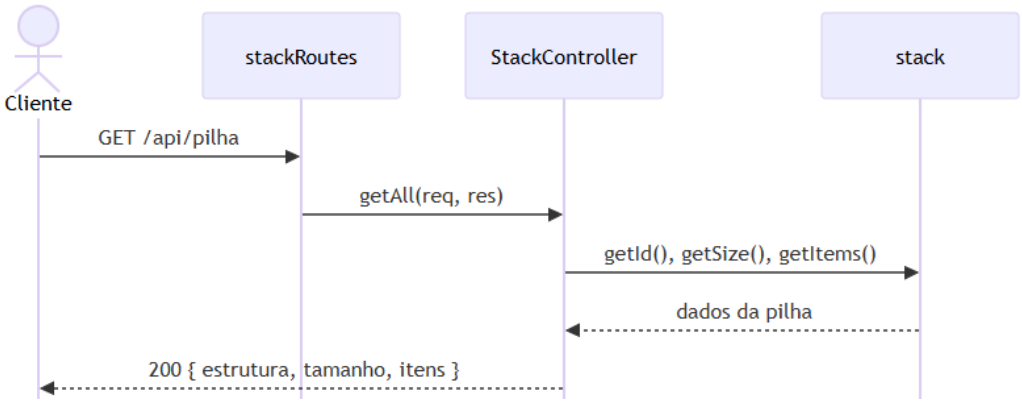
DELETE /api/pilha



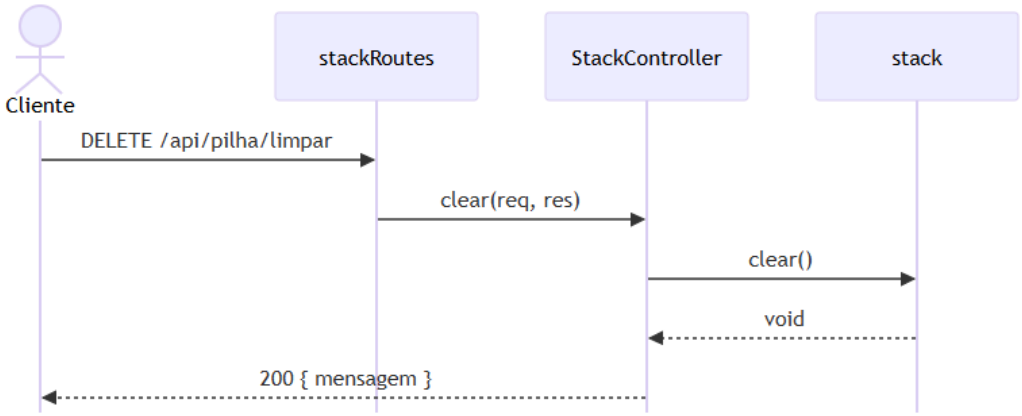
GET /api/pilha/topo



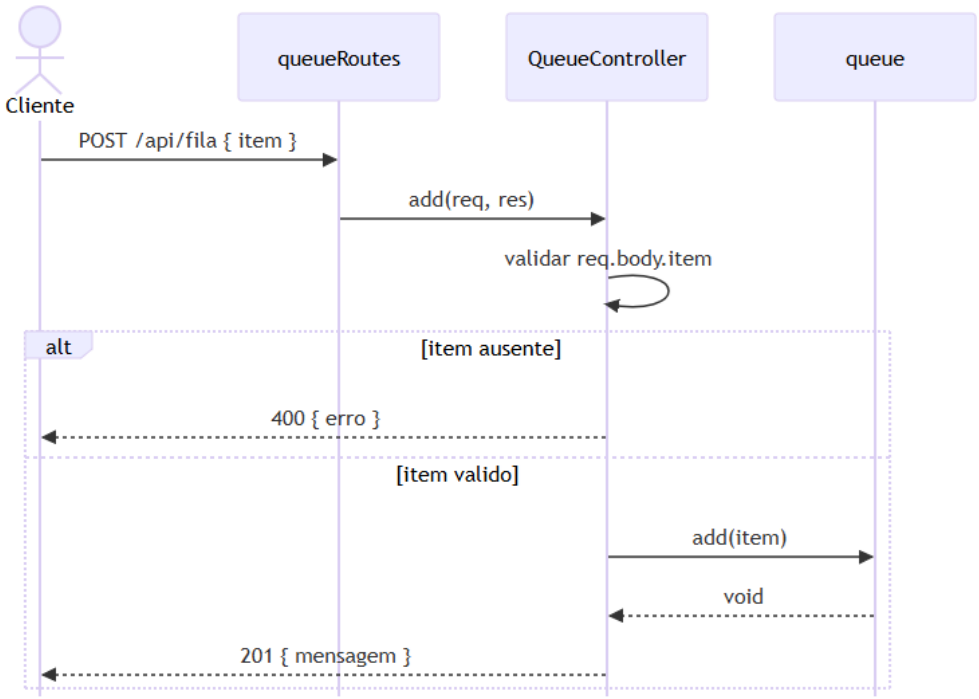
GET /api/pilha



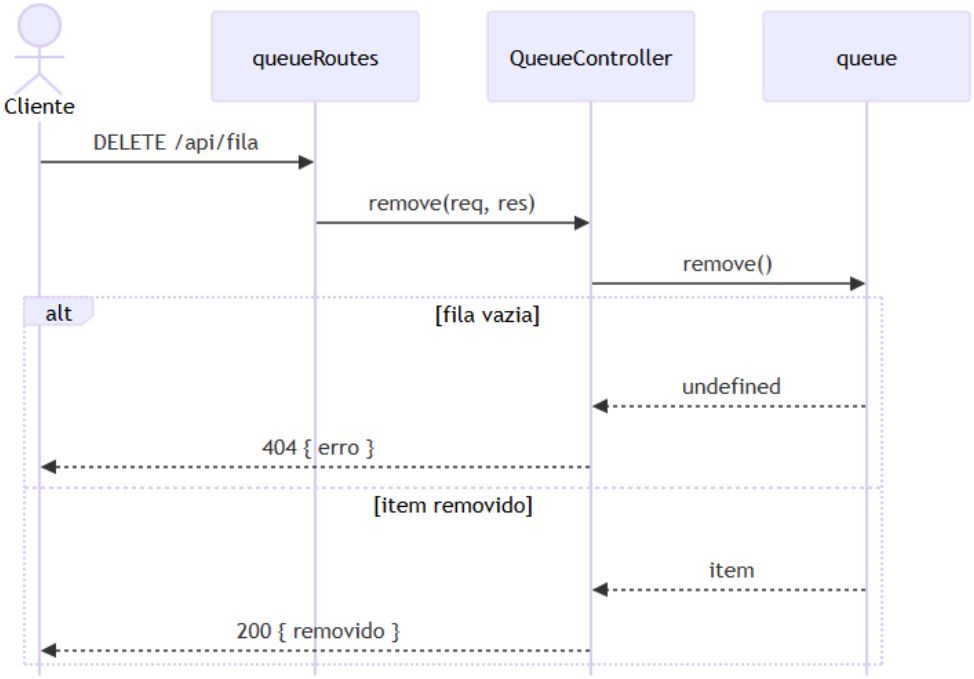
DELETE /api/pilha/limpar



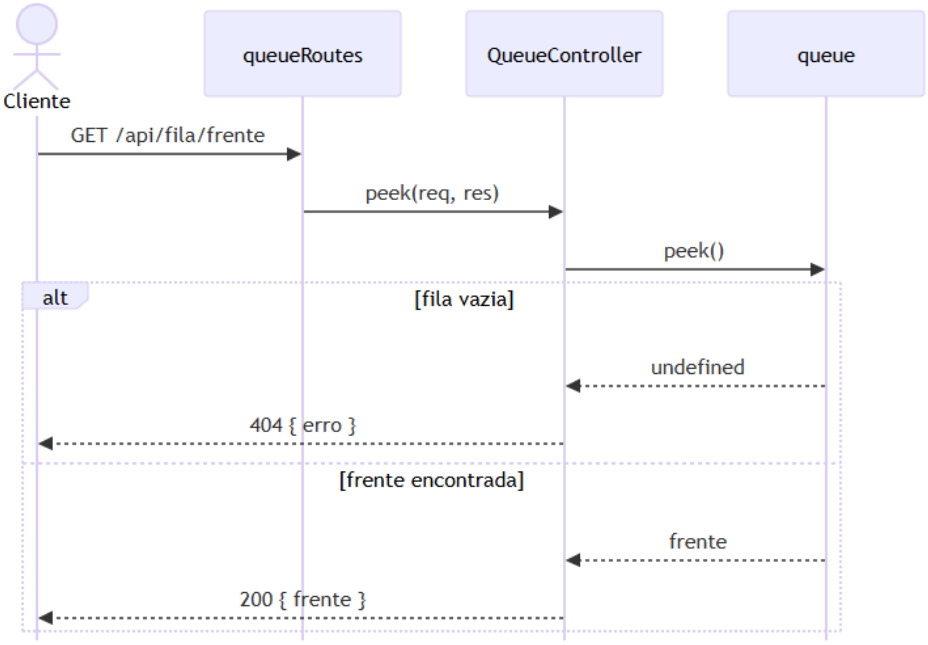
POST /api/fila



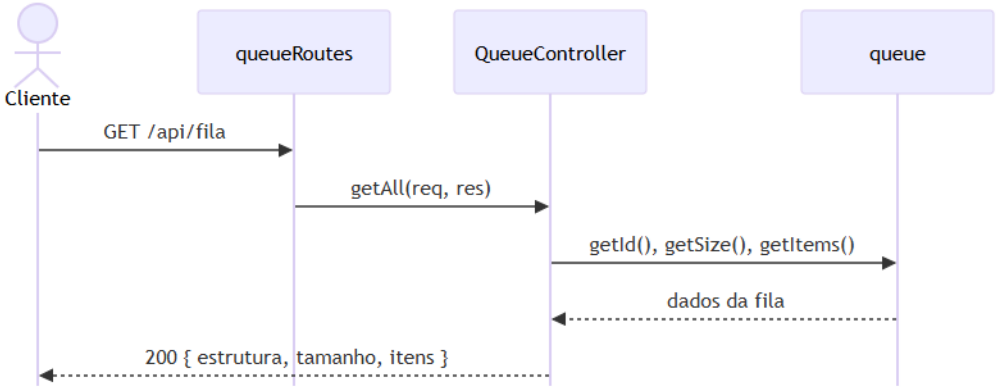
DELETE /api/fila



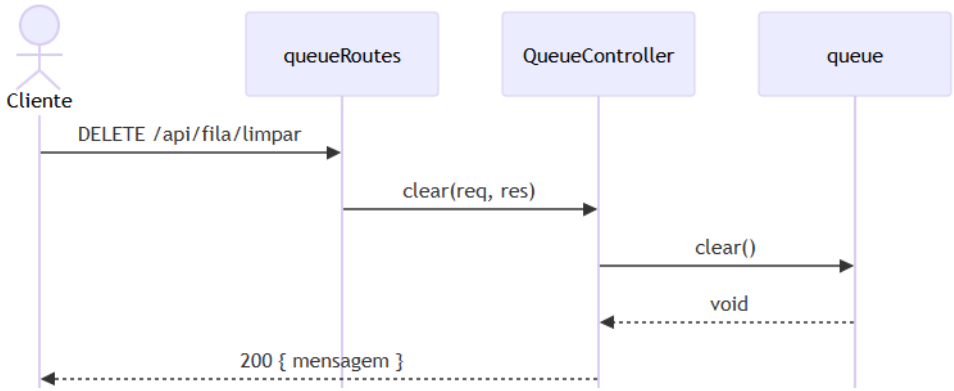
GET /api/fila/frente



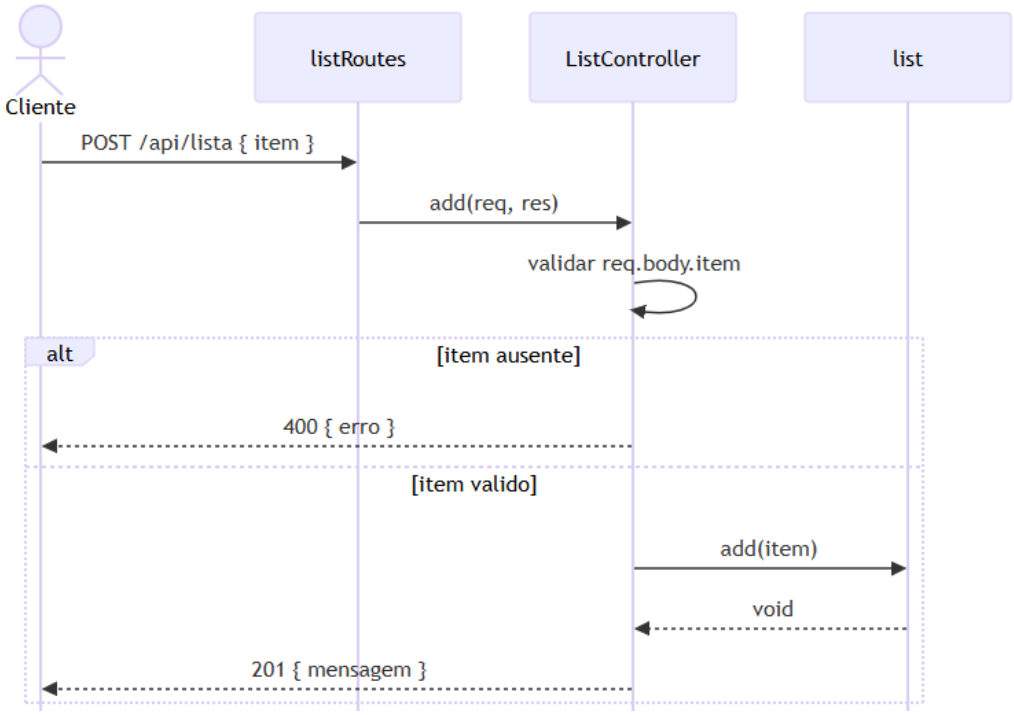
GET /api/fila



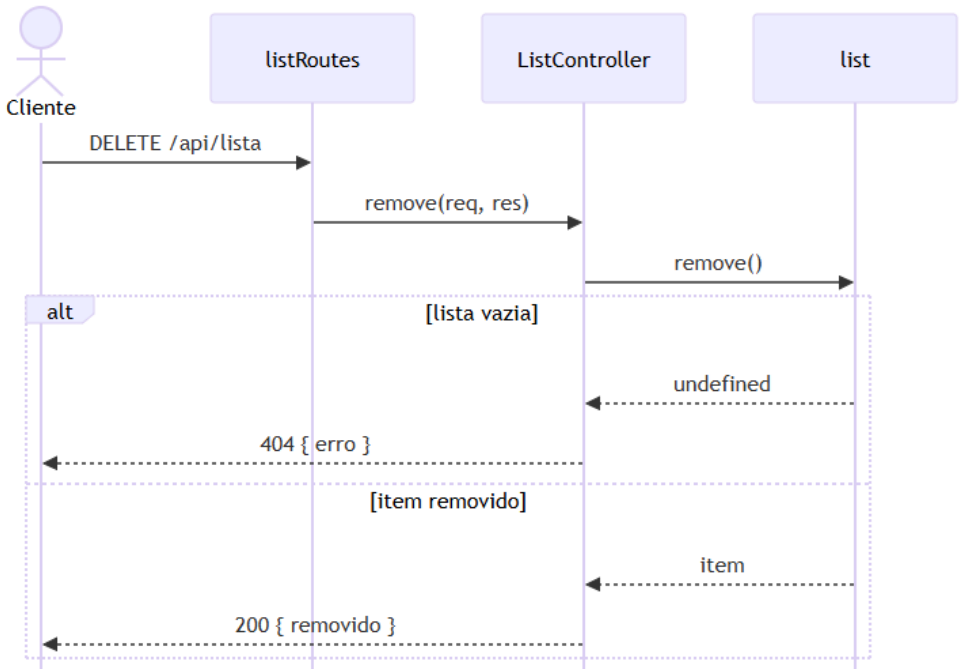
DELETE /api/fila/limpar



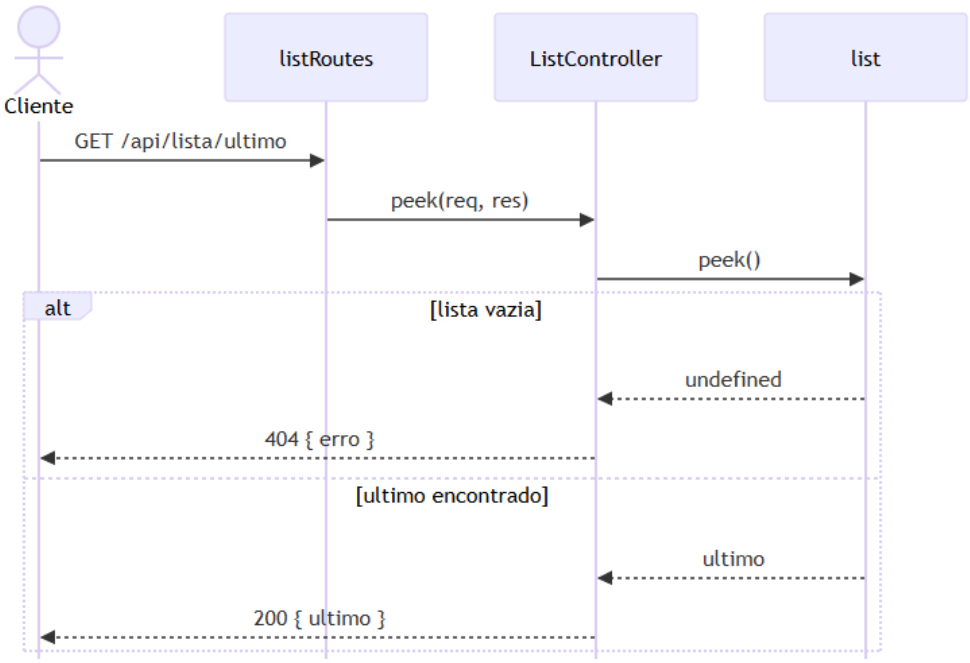
POST /api/lista



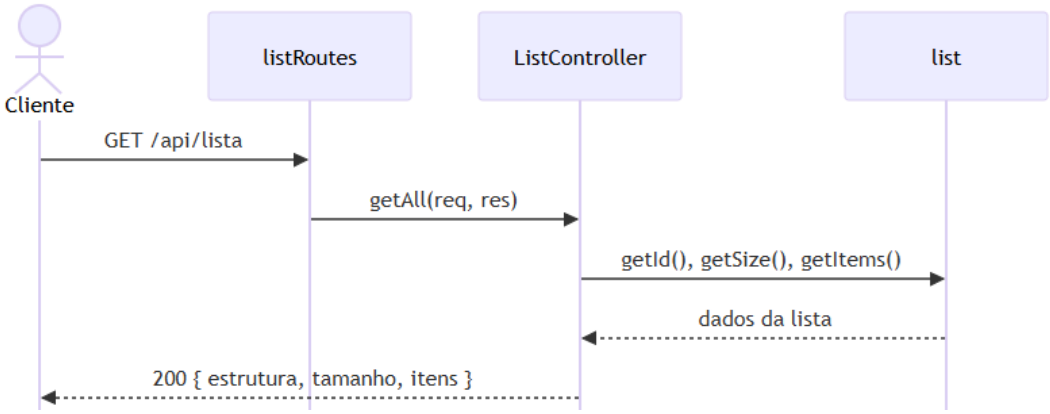
DELETE /api/lista



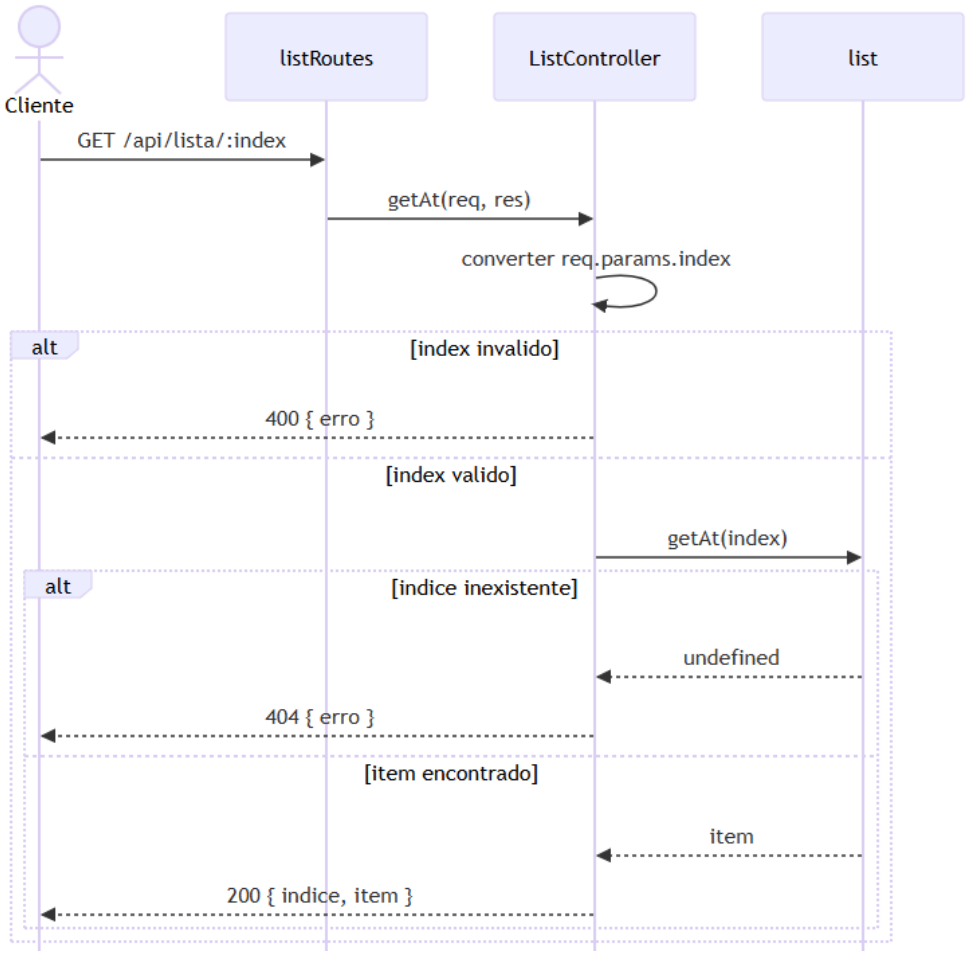
GET /api/lista/ultimo



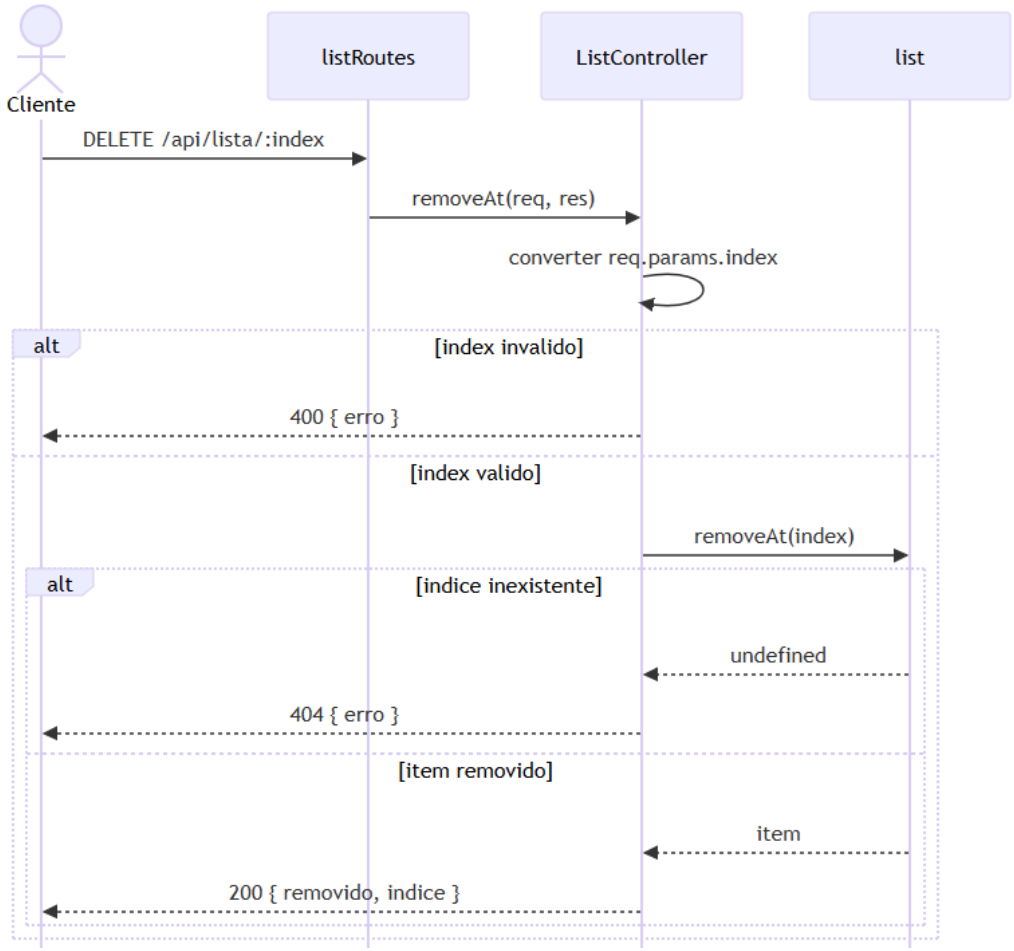
GET /api/lista



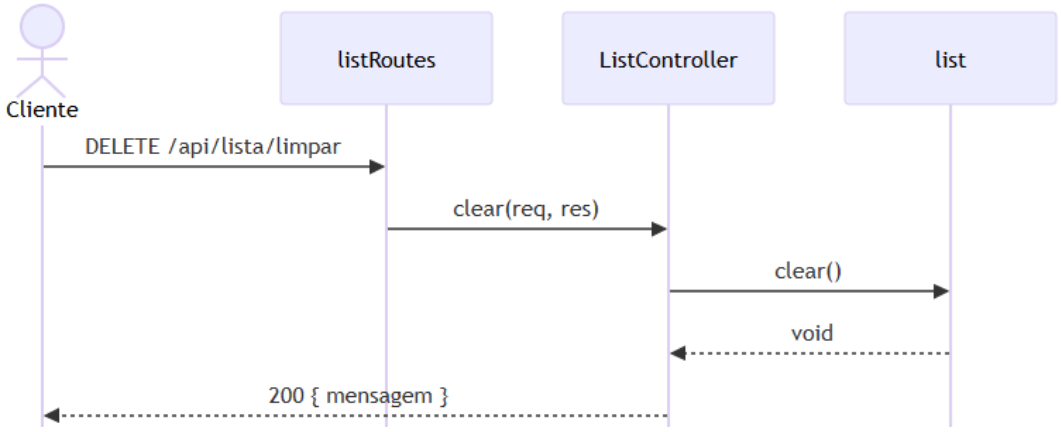
GET /api/lista/:index



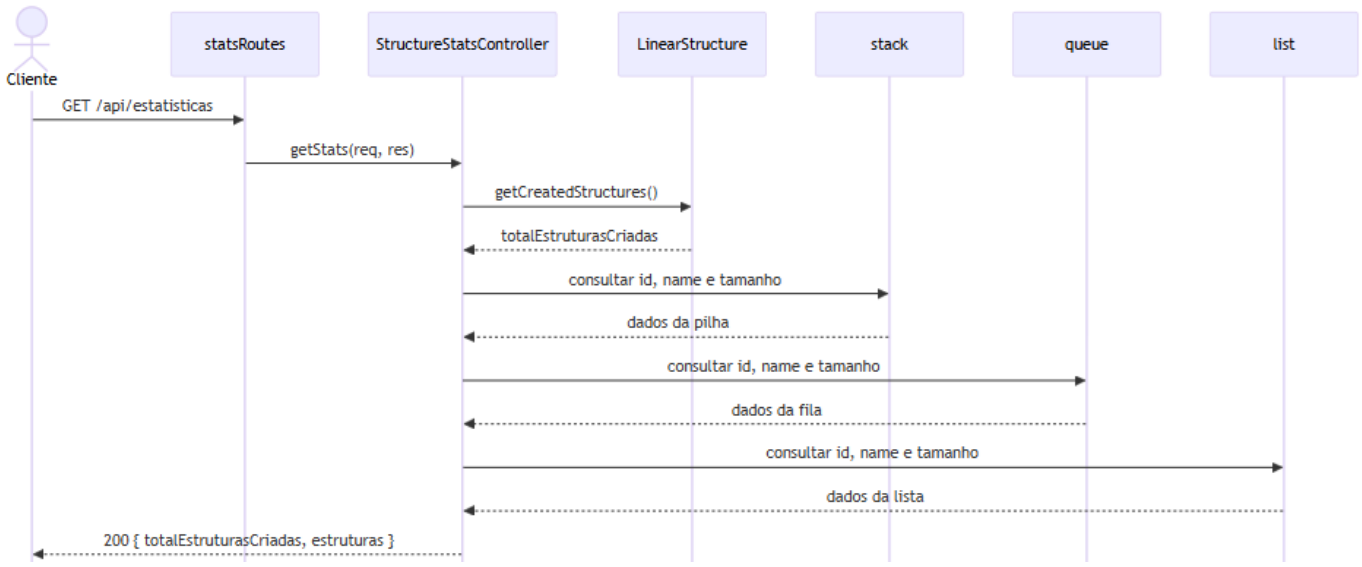
DELETE /api/lista/:index



DELETE /api/lista/limpar



GET /api/estatisticas


Cliente
statsRoutes
StructureStatsController
LinearStructure
stack
queue
list
GET /api/estatisticas
getStats(req, res)
getCreatedStructures()
totalEstruturasCriadas
consultar id, name e tamanho
dados da pilha
consultar id, name e tamanho
dados da fila
consultar id, name e tamanho
dados da lista
200 { totalEstruturasCriadas, estruturas }